



Aiden
Young

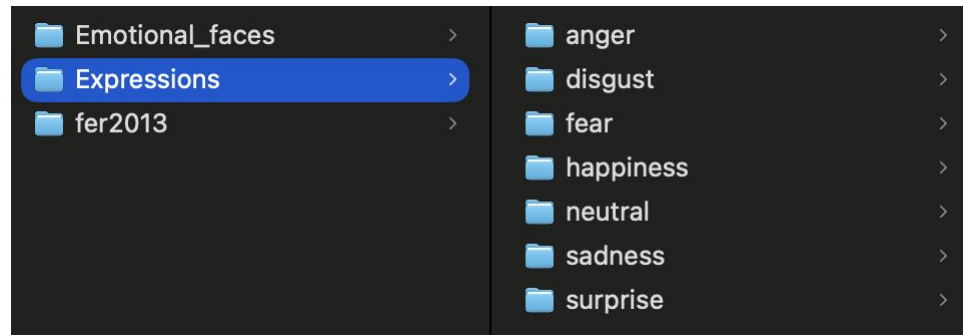
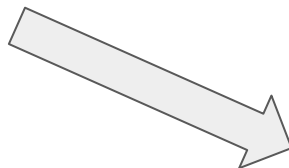
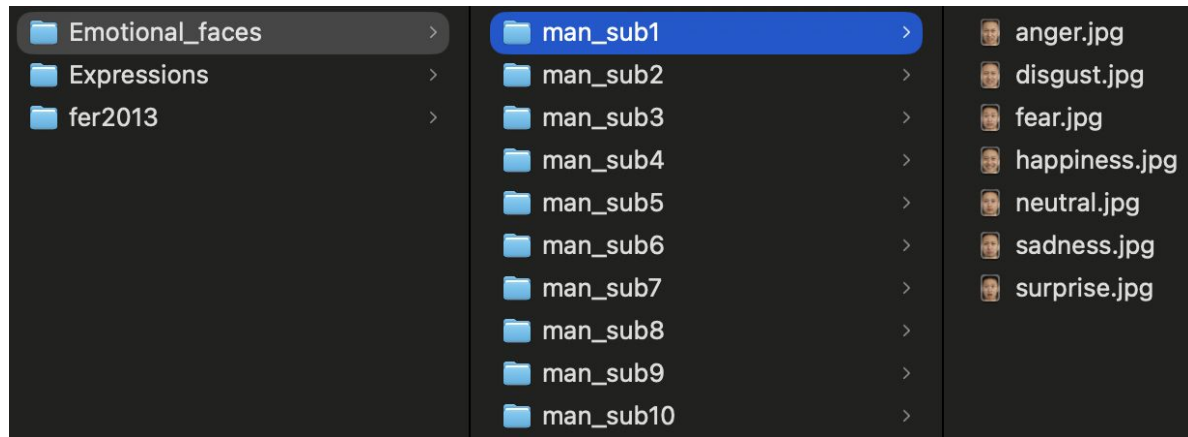
Can we build a model that effectively classifies facial expressions?

Goal: predict emotion on any face image we feed it

Use CNNs with PyTorch for modeling and the FER-2013 dataset

before we start...

First dataset (not used for actual model)



Too simple...

Only 840 images.. 120 people, 7 emotion each

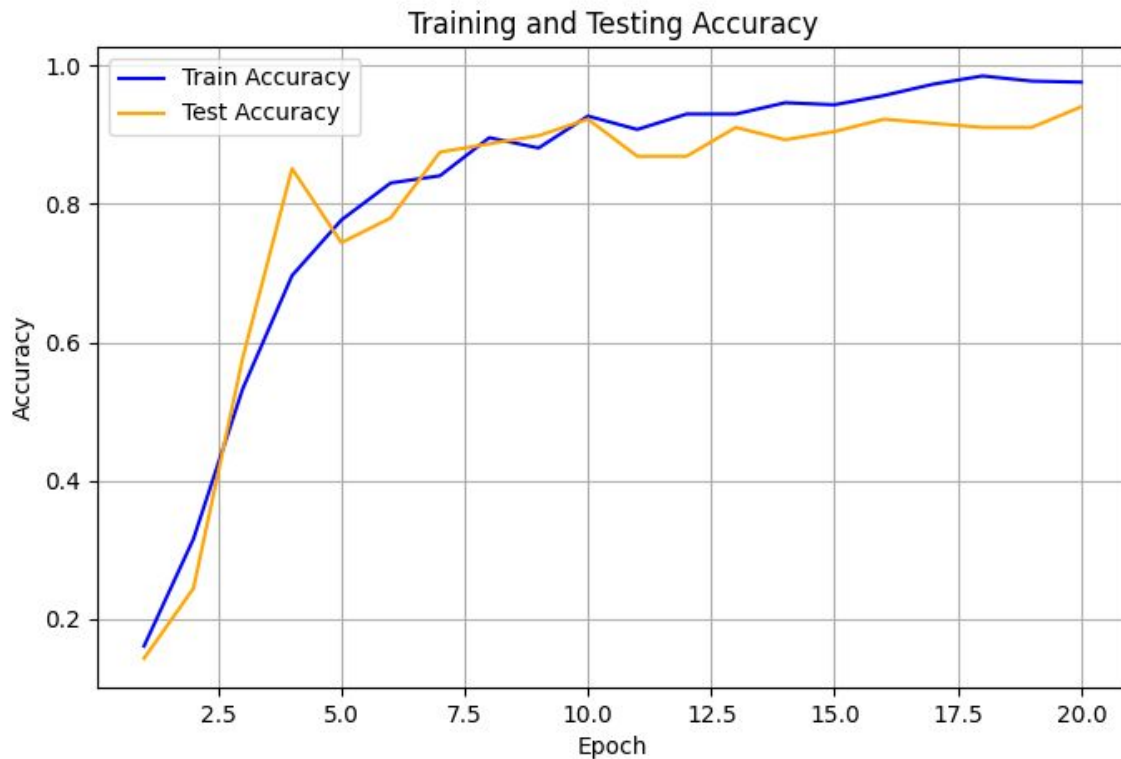
Images very uniform. No additional processing was required

Anger samples



Performance on the “toy” dataset

Up to 95% accuracy



Our Actual Dataset

- FER2013
- ~35000 jpgs
- Already split into seven categories
- Images are 48x48, 0-255 grayscale values
- “Clean” all images are useable



The screenshot shows the Kaggle dataset page for FER-2013. At the top, it says 'MANAS SAMBARE · UPDATED 6 YEARS AGO'. There are buttons for '1851', 'Code', 'Download', and a menu icon. Below the title 'FER-2013', it says 'Learn facial expressions from an image'. To the right, there are three small images labeled 'Fear', 'Happy', and 'Neutral'. Below the title, there are tabs for 'Data Card', 'Code (829)', 'Discussion (8)', and 'Suggestions (7)'. The 'Data Card' tab is selected.

About Dataset

The data consists of 48×48 pixel grayscale images of faces. The faces have been automatically registered so that the face is more or less centred and occupies about the same amount of space in each image.

The task is to categorize each face based on the emotion shown in the facial expression into one of seven categories (0=Angry, 1=Disgust, 2=Fear, 3=Happy, 4=Sad, 5=Surprise, 6=Neutral). The training set consists of 28,709 examples and the public test set consists of 3,589 examples.

Usability

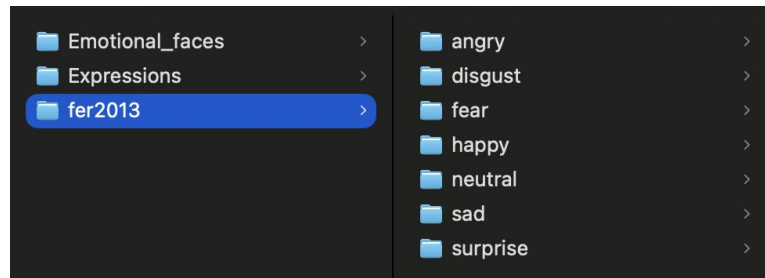
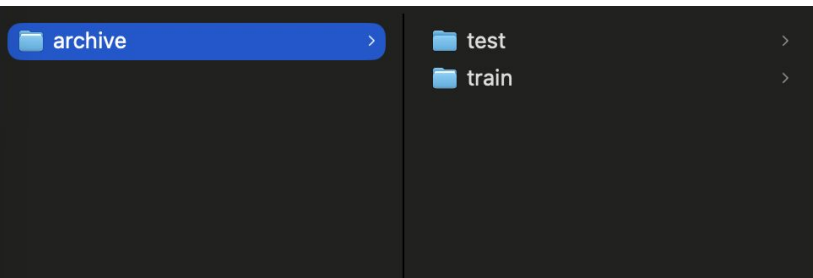
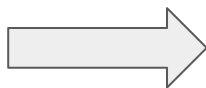
7.50

License

Database: Open Database, Cont...

Expected update frequency

Not specified



ANGER



DISGUST



FEAR



HAPPY



NEUTRAL



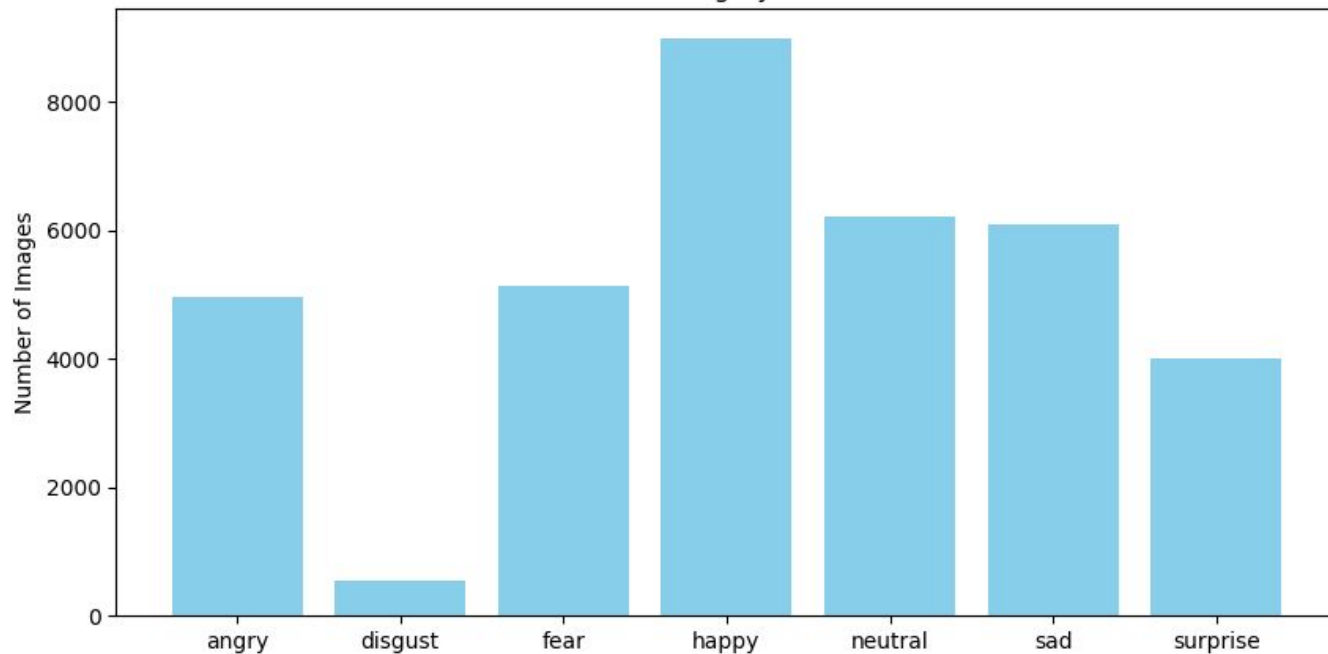
SAD



SURPRISE



FER-2013 Category Distribution



angry: 4953

disgust: 547

fear: 5121

happy: 8989

neutral: 6198

sad: 6077

surprise: 4002

Total: 35887

Needs cleaning

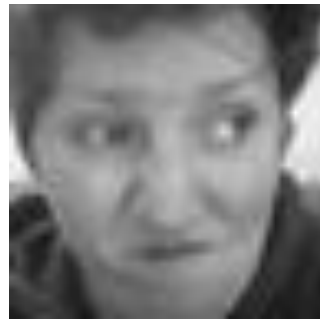
guess this one

These are happy?



what is that

It's sad



These are sad?



Top 20 Lowest Variance Faces in FER-2013

angry
Var: 0.0000



angry
Var: 0.0000



angry
Var: 0.0000



fear
Var: 0.0021



surprise
Var: 0.0000



sad
Var: 0.0000



angry
Var: 0.0000



happy
Var: 0.0024



angry
Var: 0.0000



angry
Var: 0.0000



neutral
Var: 0.0000



neutral
Var: 0.0028



angry
Var: 0.0000



neutral
Var: 0.0000



happy
Var: 0.0003



neutral
Var: 0.0030



happy
Var: 0.0000



angry
Var: 0.0000



happy
Var: 0.0013



neutral
Var: 0.0035



Removed 15 lowest variance faces images

Top 20 Highest Variance Faces in FER-2013

angry
Var: 0.1997



sad
Var: 0.1948



sad
Var: 0.1867



fear
Var: 0.1861



neutral
Var: 0.1759



sad
Var: 0.1737



sad
Var: 0.1712



sad
Var: 0.1708



surprise
Var: 0.1676



surprise
Var: 0.1676



happy
Var: 0.1672



angry
Var: 0.1671



angry
Var: 0.1671



surprise
Var: 0.1664



sad
Var: 0.1658



sad
Var: 0.1655



sad
Var: 0.1644



sad
Var: 0.1643



sad
Var: 0.1624



sad
Var: 0.1615



Many duplicates also lighting and pixel depth vary a lot

Original images: 35887



Removed low var images: 35872



Removed duplicates: 34030

Removed 1842 duplicate images

Removed 15 low variance images

Next: standardize images, darken edges,
continue to remove outliers

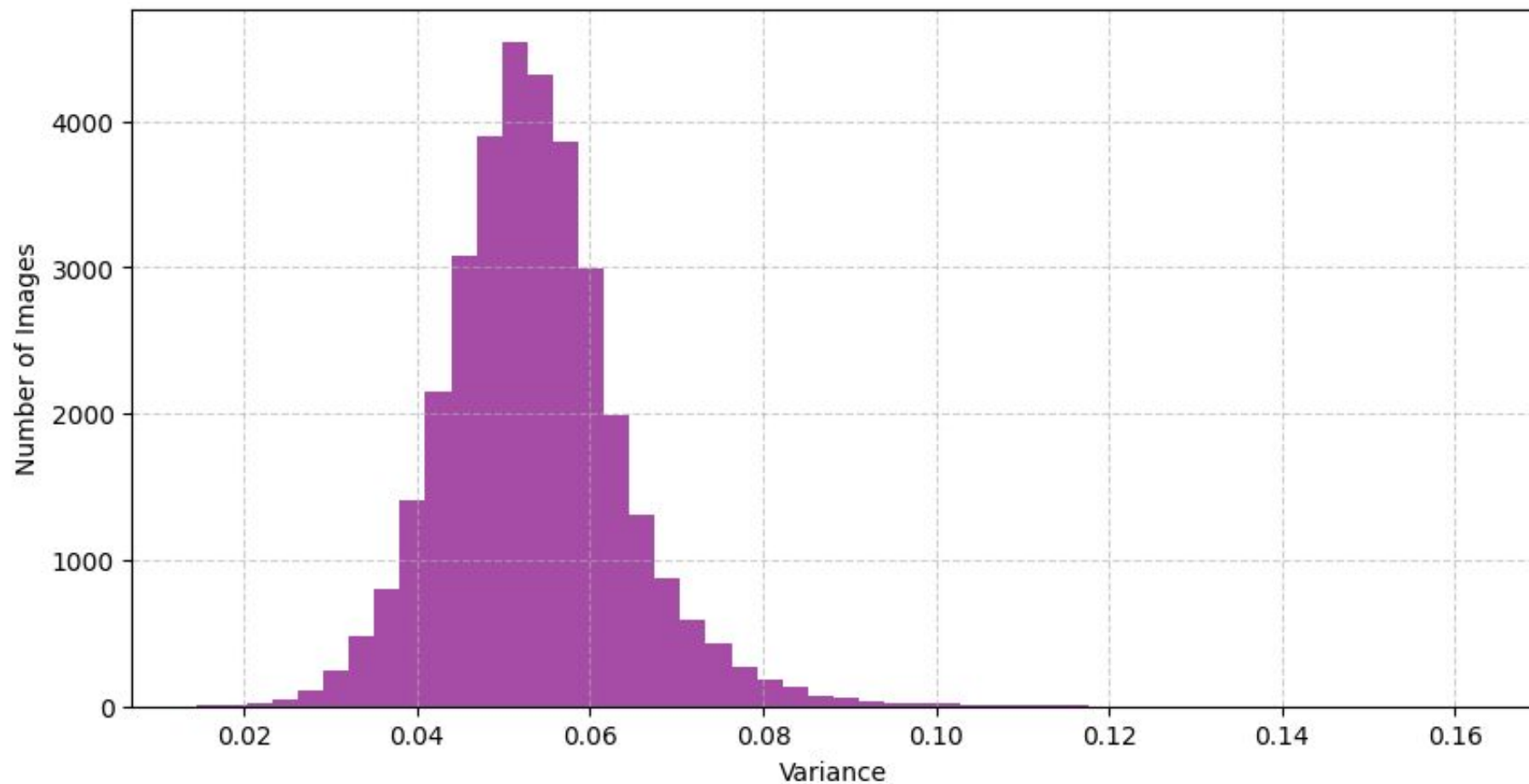
Original (Varying Contrast)



Standardized (CLAHE)



Variance Distribution AFTER CLAHE Standardization



Top 20 Lowest Variance Faces (Post-Standardization)

happy
Var: 0.0144



disgust
Var: 0.0171



neutral
Var: 0.0197



fear
Var: 0.0208



surprise
Var: 0.0145



fear
Var: 0.0184



neutral
Var: 0.0204



neutral
Var: 0.0208



angry
Var: 0.0151



happy
Var: 0.0193



surprise
Var: 0.0205



fear
Var: 0.0208



fear
Var: 0.0151



happy
Var: 0.0195



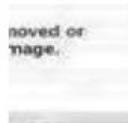
fear
Var: 0.0206



fear
Var: 0.0209



surprise
Var: 0.0153



surprise
Var: 0.0196



surprise
Var: 0.0207



happy
Var: 0.0212



Top 20 Highest Variance Faces (Post-Standardization)

sad
Var: 0.1618



angry
Var: 0.1324



fear
Var: 0.1179



sad
Var: 0.1135



angry
Var: 0.1574



sad
Var: 0.1295



happy
Var: 0.1173



surprise
Var: 0.1133



fear
Var: 0.1424



angry
Var: 0.1282



happy
Var: 0.1166



fear
Var: 0.1132



sad
Var: 0.1392



sad
Var: 0.1223



sad
Var: 0.1161



surprise
Var: 0.1118



neutral
Var: 0.1339



sad
Var: 0.1199



sad
Var: 0.1145



surprise
Var: 0.1116



Variance doesn't work... search for edges instead

Top 10 Highest Edge Density Images

Density: 0.397



Density: 0.397



Density: 0.397



Density: 0.396



Density: 0.394



Density: 0.394



Density: 0.394



Density: 0.391



Density: 0.391



Density: 0.391



Top 25 Lowest Edge Density Images

Density: 0.0000



Density: 0.0169



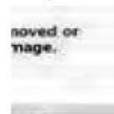
Density: 0.0417



Density: 0.0508



Density: 0.0534



Density: 0.0556



Density: 0.0642



Density: 0.0651



Density: 0.0664



Density: 0.0668



Density: 0.0677



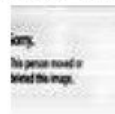
Density: 0.0677



Density: 0.0686



Density: 0.0747



Density: 0.0751



Density: 0.0773



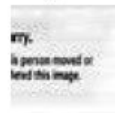
Density: 0.0777



Density: 0.0807



Density: 0.0825



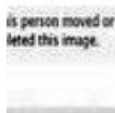
Density: 0.0981



Density: 0.1029



Density: 0.1033



Density: 0.1042



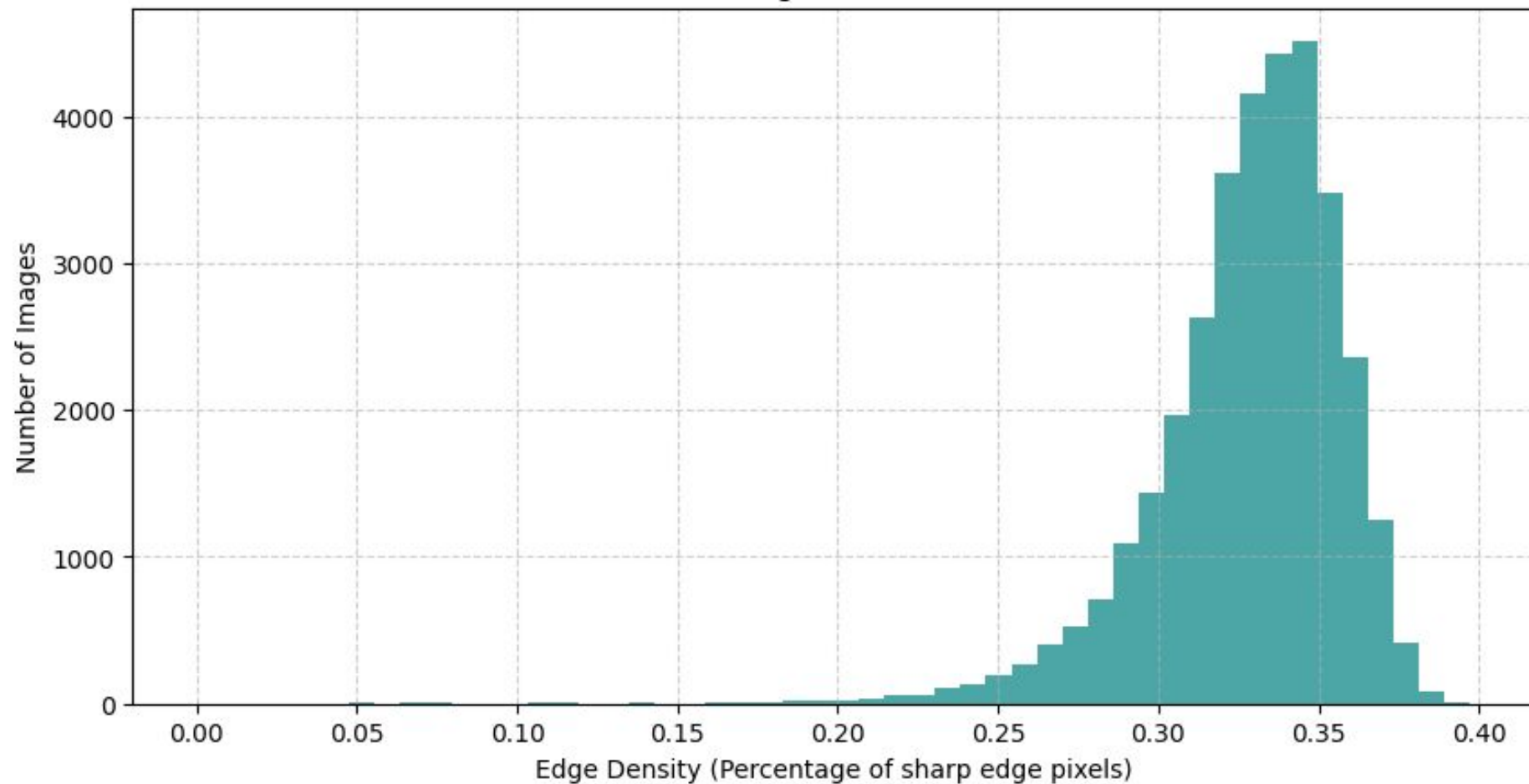
Density: 0.1050

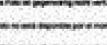
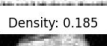
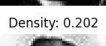
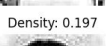
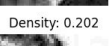


Density: 0.1094



Distribution of Edge Densities in FER-2013



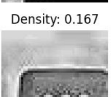
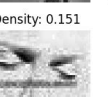
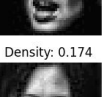
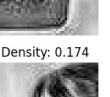
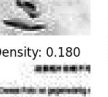
Density: 0.196 	Density: 0.180 	Density: 0.188 	Density: 0.199 	Density: 0.201 
Density: 0.185 	Density: 0.202 	Density: 0.197 	Density: 0.202 	Density: 0.207 
Density: 0.198 	Density: 0.217 	Density: 0.187 	Density: 0.204 	Density: 0.220 

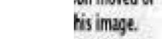
0.20: ~10% junk

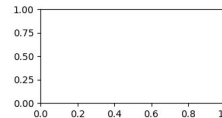
0.12: ~100%

Dataset size filter: 34030

Dataset size after filter: 33993

Density: 0.174	Density: 0.161	Density: 0.177	Density: 0.177	Density: 0.164
				
Density: 0.167	Density: 0.178	Density: 0.143	Density: 0.154	Density: 0.151
				
Density: 0.176	Density: 0.174	Density: 0.174	Density: 0.170	Density: 0.180
				

Density: 0.109 	Density: 0.137 	Density: 0.123 	Density: 0.103 	Density: 0.125 
Density: 0.103 	Density: 0.138 	Density: 0.117 	Density: 0.104 	Density: 0.115 
Density: 0.138 	Density: 0.117 	Density: 0.105 	Density: 0.118 	



Notice disgust is a lot smaller. Let's just remove it

Removed 459 'disgust' images.

Old mapping:

```
{'angry': 0, 'disgust': 1, 'fear': 2, 'happy': 3, 'neutral': 4, 'sad': 5, 'surprise': 6}
```

New mapping:

```
{'angry': 0, 'fear': 1, 'happy': 2, 'neutral': 3, 'sad': 4, 'surprise': 5}
```

angry: 4953

disgust: 547

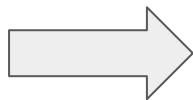
fear: 5121

happy: 8989

neutral: 6198

sad: 6077

surprise: 4002



angry: 4735

~~disgust: 459~~

fear: 4819

happy: 8791

neutral: 6040

sad: 5909

surprise: 3240

Total: 35887

Total: 33993

Preprocessing Pipeline:

Using PyTorch, Resize every image to 48x48 (in case some are different)

Convert images to grayscale (0, 255) because expression doesn't depend on color

FER-2013 is already in grayscale but preprocessing is one-time so I take the extra precaution

Convert to tensor (essentially a list of grayscale values) and also scales down to range [0,1]

```
def transform_jpg_to_dataset(src_data):  
    transform = transforms.Compose([  
        transforms.Resize((48, 48)), #nice square power of 2  
        transforms.Grayscale(), #color doesn't matter for expression  
        transforms.ToTensor() #scales images to [0,1]  
    ])   
    dataset = datasets.ImageFolder(root = src_data, transform = transform)  
  
    return dataset
```

Preprocessing Pipeline

Saved raw tensors as dictionary indexed by “X”, “y”,
“class_to_idx”

Created train-test split with sklearn with ratio 0.8 for training and 0.2 for testing.

Used a random state to keep processing consistent

Stratified labels to keep categories at equal proportion in for training and testing splits.

Saved train-test split as dictionary indexed by “train” and “test” each also indexed by “X” and “y”.

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size = 0.2, random_state = 67, stratify = y)
dataset = {
    "train": {
        "X": X_train,
        "y": y_train
    },
    "test": {
        "X": X_test,
        "y": y_test
    },
    "class_to_idx": class_to_idx
}
```

Why CNN and PyTorch?

Images are a lot of more complex so Fully Connected layers will be too complex and also overfit.

Unsupervised methods like k-means won't work because images are too different. Possible once standardized but still too complex for this project.

CNNs use convolutions to detect different patterns and features. Since each filter is only 3x3 and we have at most a few hundred filters, complexity is reduced. Once we pass images through the CNN, we can put the “feature maps” into a fully connected neural network to make predictions.

PyTorch provides optimized functions for neural networks as well as data processing. Tensorflow could work too but I have more experience with PyTorch.

Our Structure (input channels, output channels)

Conv1(1,16)-> ReLu -> Conv2(16,16) -> BatchNorm -> ReLu -> MaxPool ->

Conv3(16,32) -> ReLu -> Conv4(32,32) -> BatchNorm -> ReLu -> MaxPool ->

Conv5(32,64) -> ReLu -> Conv6(64,64) -> BatchNorm -> ReLu -> MaxPool ->

Flatten -> dropout(.5) -> fc1(64x6x6,32) -> ReLu -> fc2(32, 7 (or 6)) ->

Cross Entropy Loss -> ADAM

Why

Use ReLu because it is very simple (take max of 0 and input)

Batchnorm to keep training stable

MaxPool to reduce complexity and keep “strongest” pixels

Flatten so the 64 feature maps can be fed into fully connected neural network

Dropout to reduce overfitting

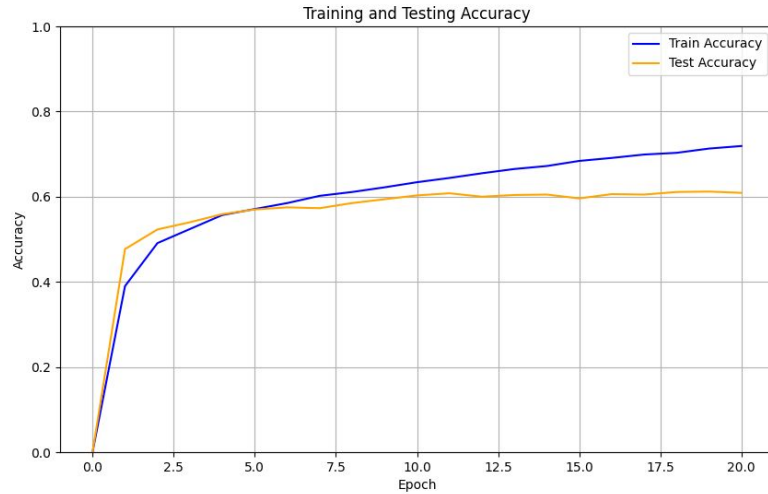
CrossEntropyLoss heavily penalizes wrong confident predictions. Generally used for classification

ADAM converges and runs faster than SGD which is the other option

Raw vs cleaned:

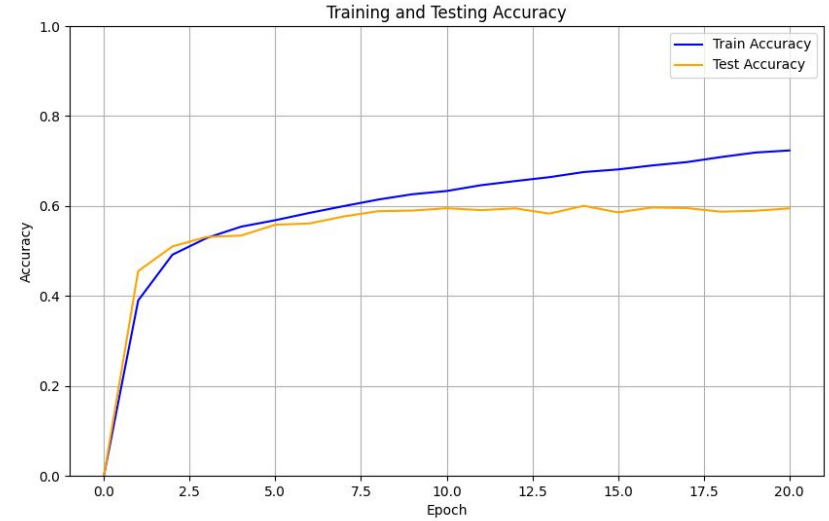
Raw tensors:

Max testing accuracy
61.2%



Cleaned data:

Max testing accuracy
%60.0



No difference between raw and cleaned

Our low variance and low edge density methods only removed ~50 bad images

Biggest difference was removing duplicates which accounted for ~4% of the dataset

Many dupes were good data as well.

Many bad images still exist. Model is slightly trained so images with highest loss could be junk or mislabelled.

Top 15 Highest Loss Training Images

True: fear
Pred: neutral
Loss: 2.72



True: fear
Pred: neutral
Loss: 1.72



True: sad
Pred: neutral
Loss: 1.45



True: fear
Pred: neutral
Loss: 2.58



True: sad
Pred: happy
Loss: 1.70



True: neutral
Pred: sad
Loss: 1.43



True: angry
Pred: sad
Loss: 2.41



True: fear
Pred: angry
Loss: 1.69



True: sad
Pred: neutral
Loss: 1.31



True: fear
Pred: happy
Loss: 2.26



True: sad
Pred: fear
Loss: 1.54



True: sad
Pred: sad
Loss: 0.87



True: fear
Pred: sad
Loss: 1.76



True: fear
Pred: sad
Loss: 1.49



True: angry
Pred: happy
Loss: 0.86



Observation:
Most are either definitely mislabelled and unreliable even for too ambiguous even for a human. Like the top three

Some are more subtle but seem to be labelled correctly like (2,2)

These images aren't junk either which was the initial expectation.

Conclusion

Yes. We created a model that can classify faces with 60% accuracy.

This is good because the expected accuracy is 17% (1/6 random guessing)

Ultimately, FER-2013 is a messy dataset with mislabelled, ambiguous, and low quality images. Even humans only score around 65% accuracy according to [this study](#)

Future goals/considerations

Use the model on webcam footage to live classify expressions

Use a more complex CNN architecture

Try to remove mislabelled or ambiguous photos similar to my method (just better)

Use other datasets with better quality and more accurate data

THANK YOU 4 LISTENING!!!!!!!